UNIVERSITY OF RWANDA
COLLEGE OF SCIENCE AND TECHNOLOGY
COMPUTER ENGINEERING
YEAR 3
MOBILE APPLICATION SYSTEM AND DESIGN
On 28th Mar 2026
Reg no:223026974

LAB 4: Consuming APIs in Flutter (Short Explanation of Each Action)

1. Setup Project

In this step, a new Flutter project is created using the Flutter CLI or Android Studio. This initializes the project structure, including folders like `lib`, `main.dart`, and configuration files.

This step is important because it prepares the environment where all UI and logic will be implemented.

---

2. Add Dependencies

You install packages like:

- `http` → to connect to the internet and call APIs

- `provider` (if used) → manage app state

Dependencies such as `http` are added in the `pubspec.yaml` file.

- `http`: Allows the application to send HTTP requests (GET, POST, PUT, DELETE) to external APIs.
- (Optional) `provider`: Helps manage state across the app.

After adding dependencies, `flutter pub get` is run to install them.

---

3. Connect to API (JSONPlaceholder)

You send a request to:

```
https://jsonplaceholder.typicode.com/posts
```

✔ Purpose: Fetch posts from a fake server

4. Create Model Class

You create a Dart class (e.g., `Post`) to represent API data.

✔ Purpose: Convert JSON → Dart objects

---

5. Create API Service

You write functions like:

- getPosts()
- createPost()
- updatePost()
- deletePost()

✔ Purpose: Handle all API communication in one place

---

6. Fetch Data using Future

API calls are asynchronous, meaning they take time.

`Future` is used to represent data that will arrive later.

`FutureBuilder` is used in UI to:

- Show loading indicator
- Display data when ready
- Handle errors

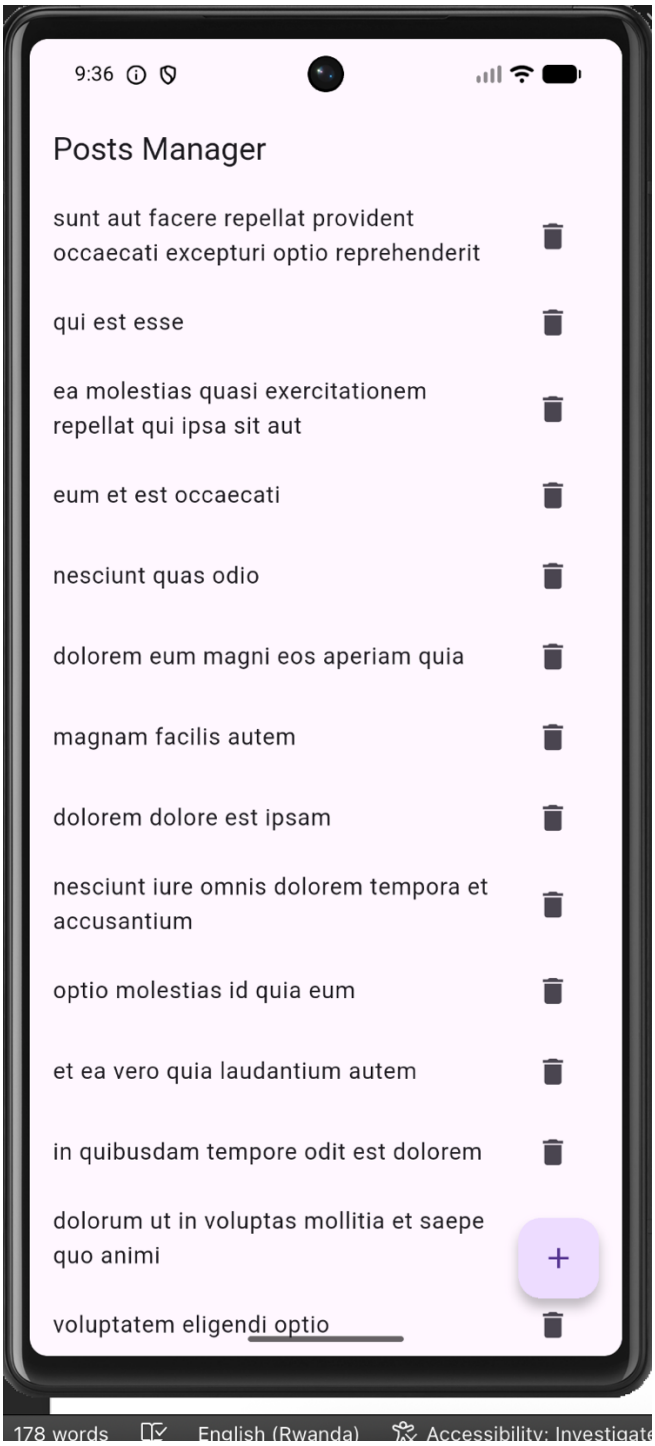
7. Display Posts List

Posts are displayed using `ListView.builder`.

Each item shows:

- Title
- Short body preview

This creates a scrollable list for users to browse posts.



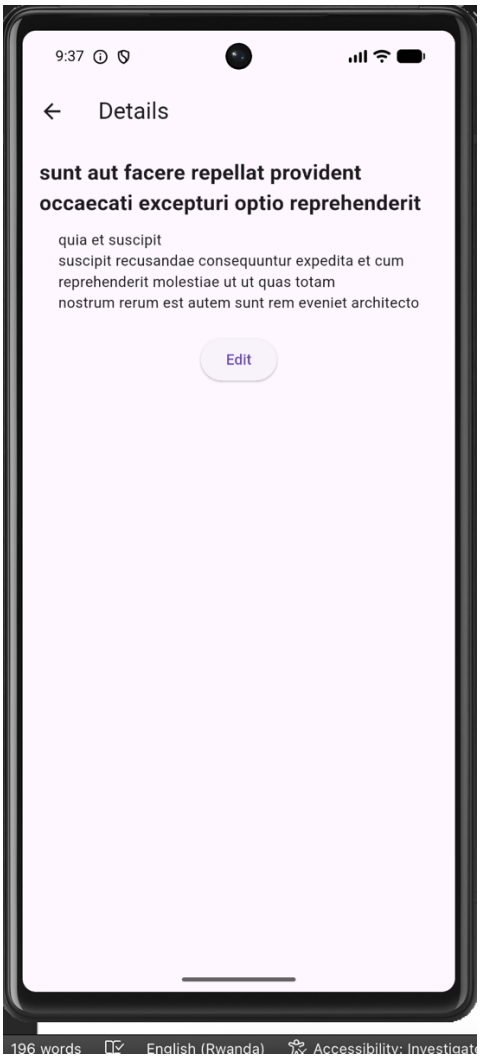
8. View Post Details

When a user taps a post:

- The app navigates to a new screen

- Displays full details (title + body)

This improves user experience by showing complete information.



9. Create New Post

9. Creating a New Post

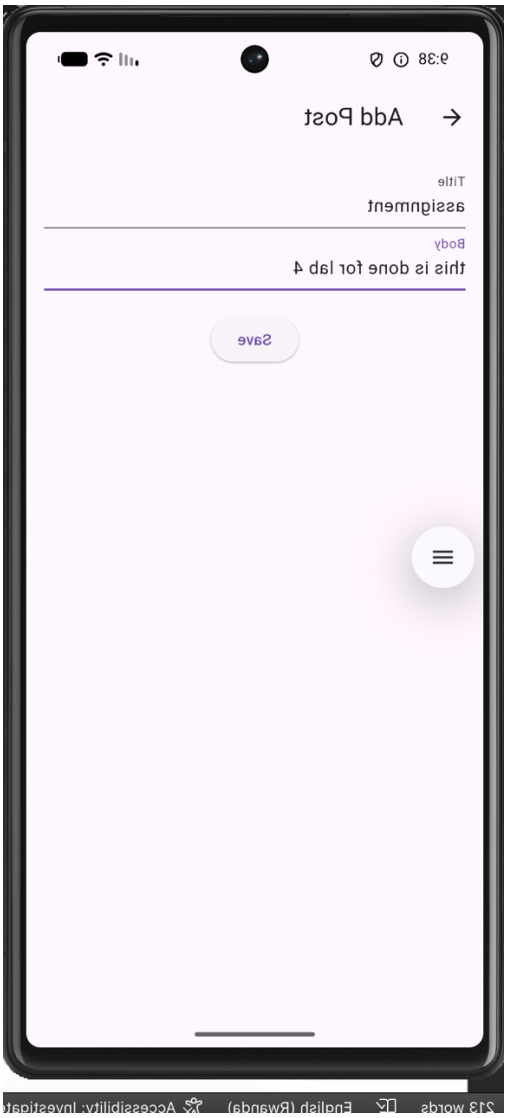
A form is created with input fields:

- Title
- Body

When submitted:

- A POST request is sent to the API
- Data is added (simulated)

This demonstrates how data is sent to a server.

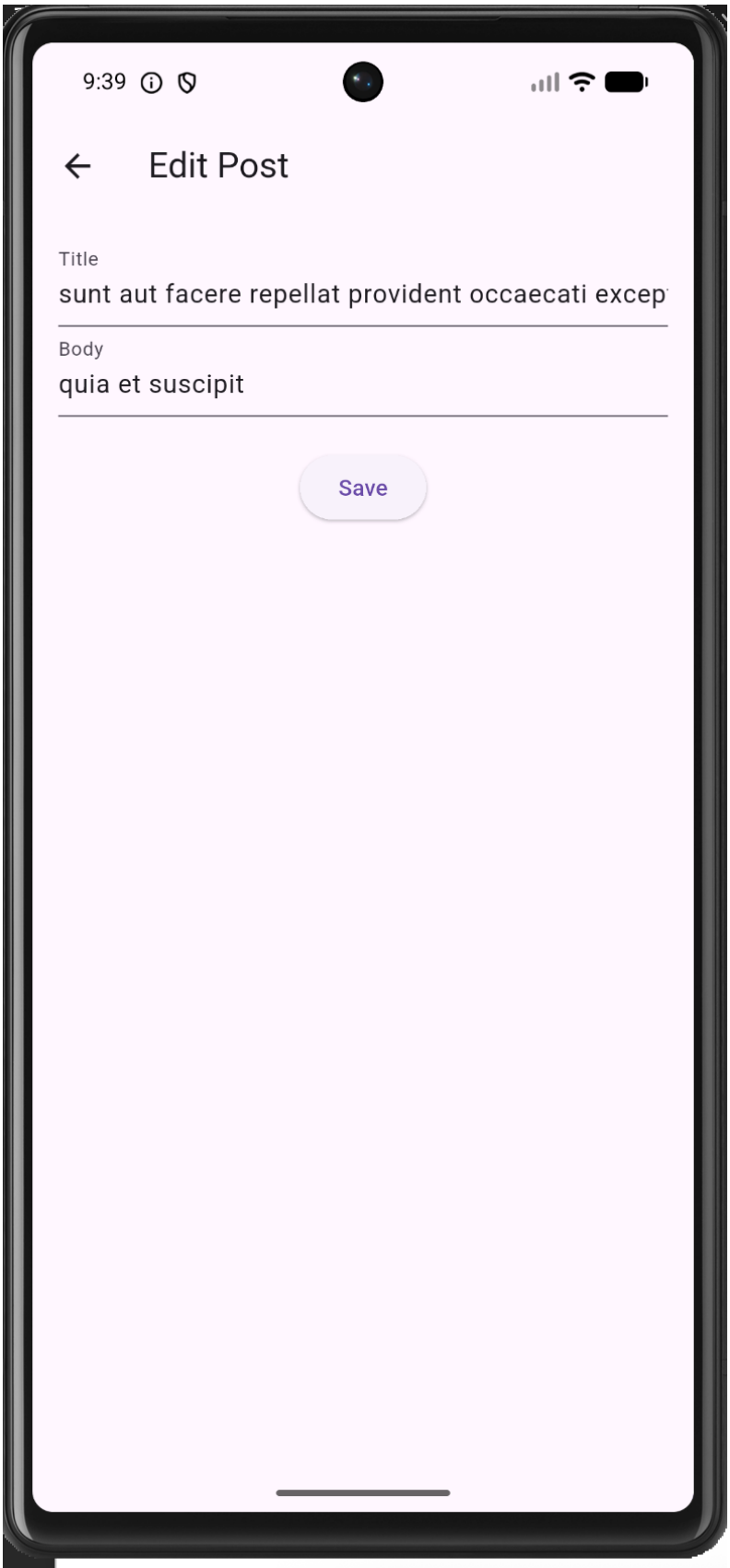


10. Edit Post

Existing post data is loaded into a form.

User modifies the content and submits:

- PUT/PATCH request is sent
- This updates the post on the server.



224 words English (Rwanda) Accessibility: Investigat

11. Delete Post

A delete button is added.

When clicked:

- DELETE request is sent
- Post is removed

UI updates to reflect deletion.

12. Handle API Errors

Handle:

- No internet
- Invalid data

✔ Purpose: Prevent app crashes

Scanned documents are below:

22 3026974

28th 03/2026
UE - CST - GAKO
II semester
Level III

MOBILE APPLICATION AND DESIGN

Introduction:

The post Manager App is a flutter application. that allows user to perform CRUD operations on post using JSON placeholder API.

API used : https://jsonplaceholder.typicode.com/posts.

features implemented.

- View all posts.
- Read post details.
- Create new posts.
- existing posts (edit)
- Delet posts

Dependencies Used in

pubsc.yaml

```
dependencies:
  flutter:
    sdk: flutter.
  http: ^0.13.6
```

"http: ^0.13.6"

purpose: this package provides the foundation for making HTTP requests to web servers and REST APIs.

Why its necessary

* Allows the flutter app to communicate with JSON Placeholder API
* provides simple methods for all HTTP operations.
  - http.get() - for fetching posts
  - http.post() - for creating new posts
  - http.put() - for updating existing posts.
  - http.delet() - for deleting posts

Why (^0.13.6):

* stable version with full support for null safety
* widely tested in production application.

why its necessary:

* Enhances the user interface with professional - looking icons.
* Used for edit, delet, add and back button

How to handle API exceptions.

1. Comprehensive Exception Handling Strategy

API calls fail for various reasons. Here's how to handle them properly

Types of Errors.

1. Network Errors.

* These errors occur when the device has no Internet connection. or very weak connection.

example: User turns off mobile or Wi-Fi → API Call fails.

2. Server Errors.

* These happen when the server (API) is not working properly
* Even if the User has Internet, the request will not succeed.

example: API returns status code like 500 (internal server Error)

3. Invalid Data Errors.

* These occur when the API returns incorrect or unexpected data.

How To Handle Errors.

→ Using try - catch.

* wrap API calls inside a try - catch block.

```
try {
} catch (e) {
    print ("Error occured");
}
```

→ check response status code

```
if (response.statuscode == 200 ){
} else {
}
```

4. The Future Widget and UI Dependencies

A Future in Dart represents a potential value or error that will be available at some time in the future. Its used for asynchronous operations like API calls

In this lab:
→ Fetching data from API takes time → so we use Future.

Why Future is Important.
* Without Future, the app would freeze while waiting for data.
* Flutter apps must stay responsive (smooth UI)

The Future Builder Widget.

Future Builder is a Flutter Widget that builds itself based on the latest snapshot of interaction with a Future

→ it automatically updates UI depending on the state of the future.

* It removes the need to manually manage loading states.

Asy ne snapshot states:

- connectionstate.none.
- connection state. waiting
- connection state. active
- connection state. done.

States of Future.

1. Waiting state
   - this happens immediately after the API calls starts
   - The data has not yet arrived

UI behaviour

* show loading spinner (circular progress indicator)

2. Complete state.
   - the API call is successfull
   - Data has been received and processed

UI behaviour

* ~~the~~ Display the Post using List view.

3. Error state.
   - something went wrong during API calls.

UI behaviour.

* show error message instead of data

Parts of UI that Depends on the Future.

UI components	Depends on	Behaviour
Loading Indicator	connection state. waiting	shows while fetching data.
Error Message	Snapshot. has Error	shows error details with retry
posts list	snapshot. has Data	Displays posts when data arrives
Empty List	snapshot. has data	shows "No posts" Message
Refresh Button	User action.	creates new Future, triggers reload.
Add / Edit Navigation.	User action	Returns to list, triggers Refresh